CHAPTER 5

GenServers: Build Cities, Not Skyscrapers

Elixir has one of the best concurrency models of any programming language today. In fact, there's a good chance that you're reading this book because you heard big claims about Elixir's ability to scale automatically across all CPU cores. The hype is real, and you're going to understand how it works by the end of this chapter.

Parallelism and concurrency have always been hot topics, but they are especially important when we're designing high-performance systems that run at low cost. A system with a high degree of parallelism is able to handle more simultaneous requests—from a web browser, an async job system, and so on—on fewer machines.

Ruby and Elixir have different concurrency models. In fact, Elixir has a concurrency model that's unlike pretty much any other mainstream language. We'll be going over those differences in this chapter, in addition to defining foundational terms that are important for you to understand.

We'll look at exactly how BEAM processes work and what makes BEAM processes so special. You'll spawn basic processes and then evolve them into GenServer processes. You'll learn that GenServers are actually not magic but are simply a well-built abstraction on top of the BEAM's process model. Finally, we'll bring it all together by thinking about how the BEAM's process model lets us build entirely different types of systems than we would build in Ruby or other languages.

But first, there's a little disclaimer that's necessary before you dive into the chapter.